

Why our HCAPO runs started writing essays:

the length runaway, explained

Agentic-RL-CA project

2026-07-30

The short version

- We trained a method called **HCAPO** (“hindsight credit assignment”) on our search-QA task. In every variant we tried, the model’s replies quietly ballooned from about **90 tokens to over 1,300** per turn — roughly ten times longer — while accuracy gained *nothing* (Figs. 1–2).
- We first traced it to a genuine **bug in our port** of the method (we divided by the wrong thing when computing the hindsight score). We fixed it, verified the fix restored exactly the behaviour the paper describes — and then watched the **fixed version blow up too**, about 40 steps later than the buggy one (Fig. 1, green).
- The root cause is simple once you see it: HCAPO judges each turn by its **average per-token score**, and averages can be gamed by padding. Hard, information-carrying tokens score low; easy filler scores high; so *a longer, more padded turn looks better than a short, dense one* (Fig. 3). Training chases that illusion until every turn is maximally padded — at which point all turns look alike, the credit signal goes silent (Fig. 4), and the method degenerates into an expensive version of plain GRPO (Fig. 5).
- Accuracy never collapsed — the tokens are *wasted*, not harmful: about **12× GRPO’s spend for less accuracy**. A deeper diagnostic of the scoring rule is running now; the candidate fixes are listed at the end.

1 What we saw

By around step 200 of the fixed run, more than half of all turns were being *cut off* by the 2048-token limit mid-sentence — the model doesn’t “finish” its thoughts, it just writes until the ceiling stops it. Training also got much slower: a step that used to take ~ 3.5 minutes now takes ~ 9 , because generating five times the tokens takes five times the time.

2 Why it happens: averages can be padded

HCAPO’s core idea is reasonable: after an episode ends, re-read each turn *knowing how the story ended*, and give extra credit to the turns that look pivotal in hindsight. To score “how good does this turn look”, it uses the model’s own per-token probabilities:

$$\text{score of a turn} = \underbrace{\frac{1}{\#\text{tokens}} \sum_{\text{tokens}} \log p(\text{token})}_{\text{average per-token log-probability}} \quad \Rightarrow \quad \text{credit ratio } \rho = \frac{\text{this turn’s score}}{\text{trajectory average}}$$

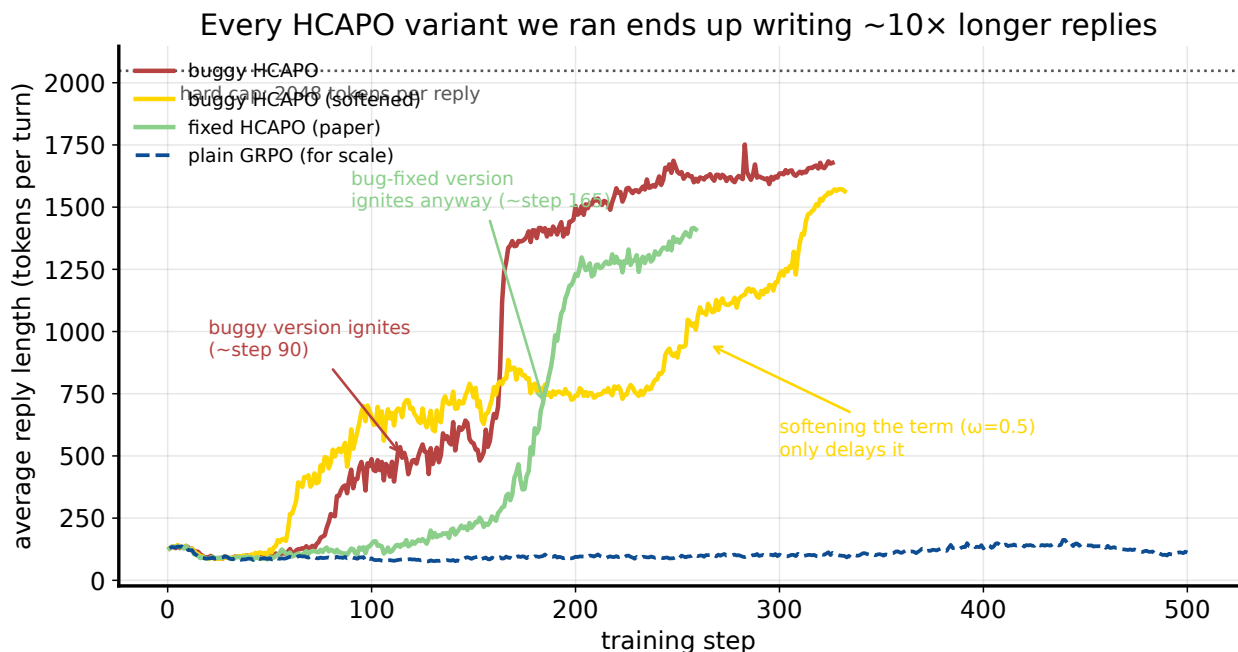


Figure 1: Average reply length over training for the three HCAPO runs, with plain GRPO (dashed blue) for scale. All three follow the same script: behave normally for a while, then take off within ~ 30 steps and slam into the 2048-token ceiling. Softening the hindsight term ($\omega=0.5$, orange) only postpones it. Fixing our implementation bug (green) also only postpones it.

The trouble is the word *average*. The tokens that carry a turn’s actual content — the new search query, the decision — are exactly the ones the model finds *hard* (low probability). The tokens that follow them — connective tissue, restatements, boilerplate — are *easy* (high probability). So the average score of a turn is not a measure of its quality; it is largely a measure of **how much easy filler it contains**:

Once the scorer prefers padded turns, reinforcement learning does the rest: the padded turns in each batch get a little more credit, the model shifts a little toward padding, and the cycle repeats. It is an arms race with no winner — which brings us to the part that surprised us.

3 We fixed a real bug — and it wasn’t enough

Our original port computed the credit ratio against the wrong baseline (the turn’s ordinary, non-hindsight probability, instead of the paper’s “average over the trajectory’s turns”). That bug was real and worth fixing: it made the ratio systematically low, half the safety clip never engaged, and it gave *global* verbosity a direct reward.

The paper’s version normalises each turn *against its own trajectory*, so if every turn gets longer together, the scores cancel and nobody gains. We predicted — in writing — that this would prevent the runaway. **It did not**, and the reason is instructive: the cancellation protects against everyone lengthening *in lockstep*, but training never moves in lockstep. In any given batch, *some* turn is a bit longer and more padded than its siblings, and that turn wins the comparison. The model is rewarded for being *longer than its own average* — a target that moves every time it is chased. Every student on a curved exam still crams; the class average just keeps rising. The race only ends when every turn is maximally padded and all scores converge — and then the signal is gone:

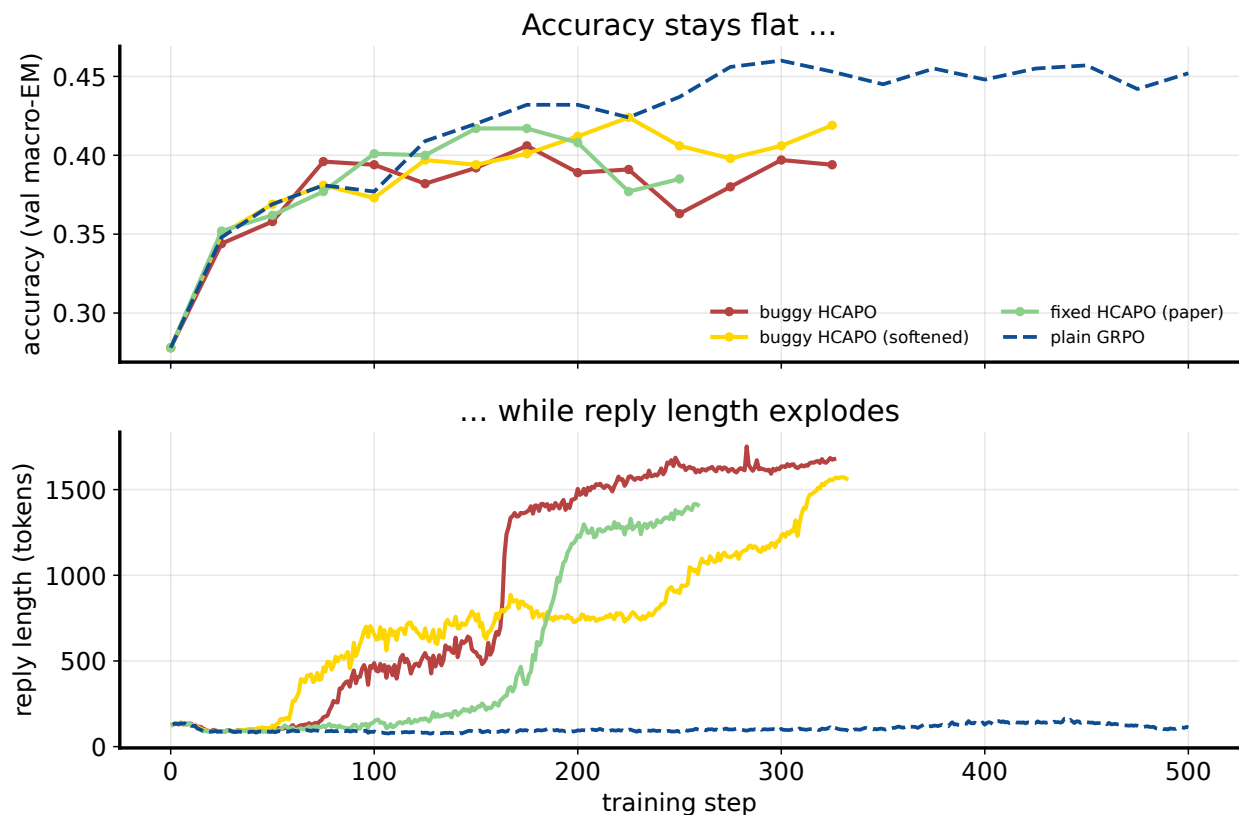


Figure 2: The extra words buy nothing. Top: accuracy on our 2,048-question validation set — flat through the entire explosion, and never better than plain GRPO. Bottom: reply length over the same steps. If the long replies contained useful extra reasoning or searching, the top panel would show it. It doesn’t.

4 The bill

5 What we’re doing about it

Running now: an offline diagnostic that re-scores logged rollouts under matched conditions (with and without the hindsight text, and with the *gold answer* injected instead) to measure exactly how much of HCAPO’s score is length/position nuisance versus genuine hindsight information — including whether the “pivotal turn” it picks even survives the controls.

Candidate fixes, in rough order of appeal:

1. **Stop averaging over a paddable denominator** — score turns on a fixed window or a length-controlled statistic, so filler cannot move the score.
2. **Exclude the final (answer) turn** from hindsight credit — the ending trivially “explains” the answer turn, a leak no normalisation removes.
3. **Inject the gold answer, not the last retrieval**, as the hindsight context — shorter, cleaner, and it cannot reward re-reading your own padding.
4. A blunt **per-turn length penalty** would mask the symptom, but leaves the mis-scoring in place — a last resort, not a fix.

HCAPO scores a turn by its AVERAGE per-token score — so padding with easy filler raises the score

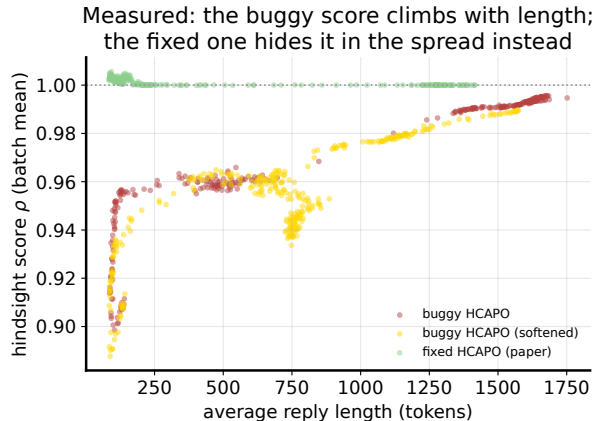
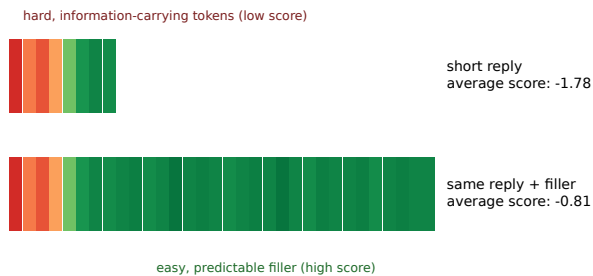


Figure 3: Left: a cartoon of the flaw. Two versions of the *same* turn — one short and dense, one padded with easy filler. The padded one has a higher *average* score, because averaging dilutes the hard tokens. Right: the same effect measured in our runs — in the buggy version the score climbs straight up with reply length; in the fixed version the mean is pinned at 1 by construction, so the same pressure shows up in the *spread* instead (Fig. 4).

Fine print

Single seed per run; all on SearchQA (4-turn episodes, Qwen3-4B, non-thinking, 2048-token cap). The HCAPO paper’s own benchmarks (ALFWorld, WebShop) use short action-style turns under tight caps, and may simply never enter this regime — our claim is about this task class, not their results. Accuracy numbers in Fig. 5 are validation macro-EM; the fixed run is still training and its final numbers will be appended at step 500. Full technical trail (timestamped pre-registrations, two retracted over-calls, and the falsified prediction): `docs/reports/2026-07-21_4b_horizon_eval/results/hcapo_wave3_preregistration.md` and the companion technical note `2026-07-29_hcapo_implementation_note/`. Rebuild: `assets/build_figures.py + tectonic report.tex`.



Figure 4: The fixed version’s runaway consumes its own signal. Green: how much the hindsight scores actually *differ* between turns (the spread of ρ) — the method’s entire information content. Gray: reply length. As padding homogenises the turns, the spread collapses from 0.135 to 0.030; by step 200 every turn scores ≈ 1.0 and the “hindsight credit” distinguishes nothing. What remains is plain GRPO plus a small position bonus — at twelve times the price.

The bill: $\sim 12\times$ GRPO’s tokens, for less accuracy (all non-thinking, same budget)

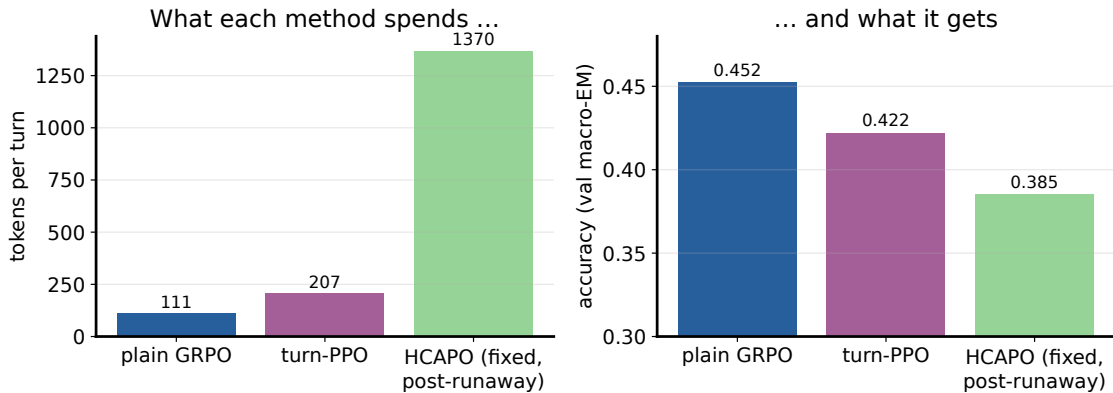


Figure 5: Same task, same budget, same non-thinking setup. HCAPO (after its runaway) spends $\sim 1,370$ tokens per turn to score *below* both plain GRPO (111 tokens) and turn-PPO (207 tokens).